Data-Intensive AI Systems

Building ML Infrastructure from First Principles

Final project report

Student number

s242796

Project: MLStore-Lite

June 10, 2026

Contents

1 Introduction

Modern machine learning systems depend on data infrastructure that can store features, update them from new events, and recover from failures. The goal of MLStore-Lite is to build a small educational prototype of such a system. The project is inspired by selected parts of *Designing Data-Intensive Applications* (DDIA), but keeps the implementation local and readable.

In concrete terms, the final project is a local ML infrastructure prototype. It contains a storage engine, leader-follower replication, sharding, batch processing, stream processing, observability, online inference, a sequential recommender, metadata, lineage, and data quality checks.

The project does not try to compete with production tools such as RocksDB, Cassandra, Kafka, Spark, Flink, or managed ML platforms. I use those systems as reference points, not as tools I claim to have mastered deeply. The purpose is to understand their architecture by implementing compact versions of central ideas: write-ahead logging, SSTable-like files, replication, sharding, batch processing, stream processing, observability, feature serving, model inference, sequential recommendation, metadata, lineage, and data quality.

The project simulates distributed ideas locally. It does not implement real network communication, consensus, automatic leader election, or production-grade fault tolerance. The AI layer is included not as a standalone model project, but to show why data infrastructure matters for ML systems: models need features, serving paths, logs, reproducibility, and traceability.

This project is also personal learning work. Coming from a mathematics background and having worked with code for a bit less than two years, I wanted to go deeper into software architecture rather than only use high-level ML libraries. The implementation was AI-assisted using Codex 5.5, Claude Sonnet 4.7, Claude Opus 4.7, and experiments with local LLMs such as Qwen models around 35B parameters. The tools helped with implementation, debugging, and writing, but the project scope and interpretation remained part of the learning process.

2 Reading Scope and Method

The reading scope focused on selected DDIA chapters rather than the whole book. The main mapping was:

Milestone	DDIA connection	Implementation
Storage	Ch. 3: Storage and Retrieval	WAL, memtable, SSTable-like files, compaction, recovery
Replication	Ch. 5: Replication	leader-follower replication, sync/async writes, failover
Sharding	Ch. 6: Partitioning	consistent hashing, virtual nodes, routing, rebalancing
Batch	Ch. 10: Batch Processing	local map-shuffle-reduce engine and batch features
Stream	Ch. 11: Stream Processing	event log, producers, consumers, offsets, windows
Integration	system composition	one object wiring storage, replication, sharding, batch, stream
Observability	operational tradeoffs	structured logs, timings, JSON-lines experiment records
AI extension	ML infrastructure	feature serving, inference, sequential recommender
Inspection layer	MLOps traceability	feature registry, quality checks, lineage records

Main reading-to-implementation mapping.

Several topics were intentionally not implemented deeply: transactions, consensus, automatic leader election, distributed commit, real network partitions, and production recovery. This boundary keeps the project focused on understanding the architecture instead of reproducing a full database or stream processor.

The AI/recommender part is designed around ecommerce event data. The code can read the RetailRocket ecommerce dataset from Kaggle if the file is placed at `data/raw/retailrocket/events.csv`. That dataset has event-style columns such as timestamp, visitor id, event type, and item id [1]. The repository does not commit the Kaggle CSV because it is external and larger than the source code. When the file is missing, the scripts use a small built-in sample with the same shape: user id, event type, item id, and timestamp. This keeps the project reproducible while still showing where a larger real dataset fits into the architecture.

3 System Architecture

MLStore-Lite is organized as a layered system:

```
storage -> replication -> sharding -> batch -> stream
        -> integration -> observability -> inference
        -> sequential recommender -> metadata/lineage/quality
```

The storage layer persists key-value data on one node. The replication layer keeps several copies of the same data. The sharding layer decides which replicated group owns each key. Batch and stream processing turn raw events into features. The AI layers consume those features and event histories. The final inspection layer explains feature meanings, validates input events, and records which data was used for predictions.

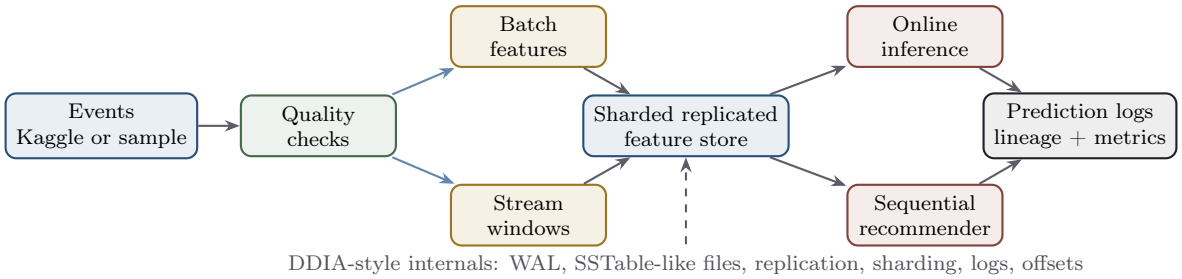

Figure 1: Final architecture of MLStore-Lite. Events become validated inputs, features, model predictions, and traceable logs.

3.1 Data Model and Key Design

The common data model is deliberately simple: the system stores keys and values. That choice makes the lower layers generic. The storage engine does not need to know whether a value is a click count, a model probability, or a lineage record. Meaning is added by the layers above it through naming conventions and metadata.

Most feature keys follow a readable pattern:

```
feature:user:<user_id>:<feature_name>
```

For example, `feature:user:42:click_count` means that the value belongs to user 42 and stores a derived click-count feature. Prediction keys and lineage records use the same idea: the prefix tells the reader what kind of object is stored, while the rest of the key identifies the user, model, or run. This is not only a coding convenience. In a partitioned system, the key is also the input to the shard router. Therefore, key design affects both human understanding and physical data placement.

3.2 How the Local System Is Set Up

The project simulates a small cluster on one machine. A node is a local storage engine backed by a directory. A replica is another node directory containing a copy of the same shard data. A shard is a logical partition of the key space, not a single file. The default topology used in the demos is:

```
2 shards
3 replicas per shard
6 local node directories
```

This local topology keeps the project runnable on a laptop while preserving the important architecture: keys are routed to shards, each shard has multiple replicas, and higher layers do not need to know which physical directory stores the final value.

4 Building the System Step by Step

The project was implemented week by week. Each week added one system idea while still reusing the layers before it. This was useful because it made the architecture easier to reason about: a later layer never appears as an isolated script, but as another piece on top of the storage and data movement layers.

4.1 Week 1: Storage Engine

The storage engine uses a write-ahead log (WAL), memtable, immutable SSTable-like files, and compaction. A write is appended to the WAL before in-memory state is updated, so the node can recover after restart. Recent writes live in the memtable, and older flushed data lives in sorted files. Compaction merges those files and removes overwritten values.

The DDIA idea behind this part is that a database is not only a dictionary from keys to values. It also needs a physical strategy for writing to disk, recovering after crashes, and keeping lookup cost reasonable as data grows. The WAL gives durability, the memtable gives fast recent writes, and SSTable-like files make disk data immutable and easier to merge. This is a simplified LSM-tree style design, inspired by the same family of ideas used by systems such as LevelDB and RocksDB.

4.2 Week 2: Replication

Replication uses a leader-follower model. Each replicated group has one leader and two followers, giving replication factor three. Writes go to the leader and are copied to followers. The project also supports manual failover by promoting a follower.

This implementation does not claim to solve consensus. It is closer to a small laboratory version of leader-based replication from DDIA: one node accepts the write first, followers copy the result, and reads can still work after one node is removed from service. The useful lesson is the separation between a logical record and its physical copies. The value for a key is one logical fact, but the system may store that fact on several nodes for fault tolerance.

4.3 Week 3: Sharding

Sharding divides different keys across replicated groups. MLStore-Lite uses consistent hashing, where each key is hashed onto a ring and routed to the next shard. This avoids manually assigning key ranges and makes rebalancing easier when shards are added. In the demos there are two shards, and each shard has three local replicas.

This is where partitioning and replication meet. A shard is not one file; it is the ownership of a subset of keys. A replica is not a different partition; it is another copy of the same shard data. In this project, a key such as `feature:user:42:click_count` is routed to one shard, and that shard is stored by three local node directories. The consistent hash ring decides the owner, while the replication layer decides how many copies exist inside that owner group.

4.4 Week 4: Batch Processing

The batch engine follows a small MapReduce pattern:

```
map -> shuffle -> reduce
```

Raw historical events are mapped into intermediate feature pairs, grouped by feature key, and reduced into final values. Example stored keys are:

```
feature:user:42:click_count
feature:user:42:purchase_count
```

The important idea is derived data. Raw events are the original observations, while features are computed summaries that models can use. For example, many old click and purchase events can become a small number of per-user counts. The batch layer is useful when the input is finite: a file, a historical dataset, or all events up to a chosen time. It is intentionally small, but it keeps the central MapReduce shape from DDIA: map local records, shuffle by key, then reduce each group.

4.5 Week 5: Stream Processing

The stream layer handles newer events. Producers append events to a local event log. Consumers read records and store offsets. A stream processor groups events into tumbling windows and writes recent feature updates to the same sharded replicated store. This is a small local version of ideas found in Kafka, Flink, and Kafka Streams.

The difference from batch processing is the role of time. In batch processing, the dataset is already there. In stream processing, new records keep arriving, and the system must remember how far it has read. This is why offsets matter: an offset is the consumer's position in the event log. Tumbling windows then group recent records into fixed time intervals before writing feature updates.

4.6 Week 6–7: Integrated Feature Store

After storage, replication, sharding, batch, and stream processing existed separately, the integration layer connected them into one object. This matters because infrastructure value comes from composition. A batch job should not only print feature values; it should write them to the same store that stream processing and inference will read from. The integrated feature store therefore acts as the bridge between data engineering and ML serving.

The integrated layer also makes the read/write path more realistic. A client can ask the system to write a feature without manually choosing a node directory. The request goes through routing, lands on the correct replicated group, and is then persisted by the underlying storage engine. That path is small in code, but it contains the core idea behind many data platforms: separate the user-facing API from the physical placement of data.

4.7 Week 8: Evaluation and Observability

Week 8 adds observability: structured logs, timing measurements, and JSON-lines experiment records. The evaluation script measures batch feature runtime, stream production, stream processing, feature reads, and shard distribution. These measurements are not production benchmarks. They are local evidence that the system runs and can be inspected.

This part connects the project to MLOps debugging and logging ideas. The goal is not to create a monitoring platform. The goal is to make runs reproducible: if a demo produces a timing number or a prediction, there is a small JSON-lines record showing what happened, when it happened, and which parameters were used. That makes the system easier to debug and easier to discuss in the report.

4.8 Week 9: Online Inference

Week 9 adds online feature serving and inference. A feature server reads stored features for a user, a deterministic purchase-intent model computes a purchase probability, and predictions are written to a JSON-lines log. The model also reports confidence and warnings when expected features are missing.

The key architectural point is that the model does not own the data pipeline. The model consumes features that previous layers have prepared. This makes the AI layer a client of the infrastructure, not a replacement for it. In practical ML systems this distinction is important: model quality depends not only on the formula used for scoring, but also on whether the features are available, consistent, and recent enough for the decision being made.

4.9 Week 10: Scaling Experiments and Cloud Design

Week 10 added local scaling experiments and a cloud architecture design note. The workload experiment increases the number of events and records how runtime changes. The shard hotspot experiment shows what happens when many keys route to the same shard. These are still laptop-scale observations, but they make the limitations visible instead of hiding them.

The cloud design document explains how the same conceptual layers could map to managed services: durable object storage, managed streaming, distributed compute, containerized services, observability, and model serving. Nothing in the project requires cloud infrastructure, but the design note helps connect the local code to realistic production architecture.

4.10 Week 11: Sequential Recommendation

Week 11 adds a small Transformer-style sequential recommender. Instead of only using counts, it looks at ordered user behavior such as:

```
view laptop -> view laptop -> add_to_cart laptop
```

Events and items become tokens, tokens become numeric IDs, a small attention mechanism weighs recent history, and the model outputs a purchase probability. This is not a production deep-learning system. It is an educational version of sequence-based recommendation.

The intended larger input for this layer is the RetailRocket Kaggle ecommerce events dataset. Its event rows can be converted into ordered user histories: `visitorid` becomes the user, `event` becomes actions such as view, add-to-cart, or purchase, `itemid` becomes the item token, and `timestamp` orders the sequence. In the submitted repo, if that Kaggle file is not present, the same code path uses the built-in sample so that the training and demo commands still run on any machine.

4.11 Week 12: Metadata, Lineage, and Data Quality

Week 12 adds metadata, lineage, and data quality. The feature registry explains stored feature keys. The quality layer validates events before processing. The lineage log records which feature keys or event histories were used for a prediction. The idea is simple:

```
quality checks protect the input
feature registry explains the stored data
lineage explains the prediction
```

This final layer is not about making the model more complex. It is about making the system more understandable. A prediction is easier to trust when the feature definitions are documented, invalid input events are rejected, and the system can show which features or event sequences contributed to the output.

5 End-to-End Flow

The final system can be read as one pipeline. Historical events are processed by the batch layer. Recent events are appended to the stream log and processed in windows. Both paths write derived features into the same sharded, replicated feature store. The inference layer reads those features, and the sequential recommender can also read ordered event histories. Outputs are written as prediction records and lineage records.

```
historical events -> batch features
recent events     -> stream windows
features          -> sharded replicated store
store + histories -> inference and recommender
predictions       -> logs, lineage, quality reports
```

The important point is that the AI layer is downstream of the data system. A model score is not just a mathematical function. It depends on whether events were accepted by quality checks, whether features were computed, whether the right shard served the data, and whether the prediction was logged for later inspection. This is why the project combines DDIA-style infrastructure with MLOps-style observability.

6 Results and Reproducibility

The project can be checked locally with one command:

```
make all
```

This runs tests, compile checks, Week 8–12 demos, and the final demo. Docker is also supported:

```
make docker-build
make docker-run
```

The latest local verification passed with 42 tests. The Week 12 demo reported 10 registered features, 3 valid events, 2 intentionally invalid events, 6 batch features written, and 1 lineage record. The final demo writes prediction logs, lineage logs, and quality reports under `demo_data/`. Generated files are ignored by git and are meant for inspection, not as source code.

For the recommender experiment, the data source is explicit in the output. If `data/raw/retailrocket/events.csv` exists, the run uses the RetailRocket Kaggle events. Otherwise the output says that the source is the built-in sample. In the current repository state the built-in sample is the reproducible default, while the Kaggle path documents how to repeat the same pipeline with a larger external ecommerce dataset.

Check	**Observed result**	**Meaning**
Test suite	42 tests passed	Core behavior is covered by automated checks.
Week 8 evaluation	JSON-lines metrics written	Runs are inspectable and reproducible.
Week 11 recommender	Model artifact and prediction log	The AI layer can consume ordered behavior.
Week 12 inspection	10 registered features and 1 lineage record	Feature meanings and prediction inputs are traceable.
Docker run	Tests and demos pass in container	The project is easier to run outside my local machine.

Representative local verification results.

These results should be interpreted as functional validation, not as a claim of production performance. The project checks that the layers work together, that records are written where expected, and that generated artifacts can be inspected. The absolute timing values are laptop-dependent and are less important than the fact that the system exposes them.

7 Comparison With Production Systems

The main difference is scale and operational complexity. Production systems run across real machines, handle concurrent users, recover from many failure modes, and optimize for performance. MLStore-Lite runs locally and prioritizes readability.

This comparison was useful because it gave each local component a reference point. For example, the storage engine made LSM-tree terminology concrete, the stream layer

MLStore-Lite layer	Reference	Shared idea
Storage engine	RocksDB / LevelDB	WAL, memtable, SSTables, compaction
Sharding and replication	Cassandra	partitioned key space, replicated data
Event log	Kafka	append-only events, producers, consumers, offsets
Batch processing	Spark / MapReduce	map, shuffle, reduce over finite data
Stream processing	Flink / Kafka Streams	continuous event processing and windows
Integrated workflow	Databricks / platforms	end-to-end data and ML infrastructure
Inference and recommender	feature stores / recommenders	feature retrieval, prediction logging, event sequences
Metadata and quality	ML observability tools	feature definitions, checks, lineage

Table 1: Conceptual comparison with production systems.

made offsets and event logs concrete, and the recommender showed why feature stores exist in ML workflows. I did not download or run these production systems. They were used as architectural comparisons, while the implemented system stayed small enough to understand directly.

8 Design Tradeoffs

Several choices were made to keep the project educational. First, all nodes are local Python objects rather than real network services. This removes network setup, but keeps the concepts of nodes, replicas, leaders, and shards visible. Second, the project uses simple JSON and local files rather than external databases. This makes recovery logs, experiment logs, and prediction logs easy to open in a text editor.

Third, the AI models are intentionally lightweight. The purchase-intent scorer is deterministic, and the sequential recommender is a small attention model rather than a full PyTorch training stack. This keeps the focus on the infrastructure question: how do events become features, how are features served, and how can predictions be traced? A larger model could improve ML realism, but it would also make the repository harder to inspect.

Fourth, the project favors explicit scripts over hidden automation. The Makefile provides one-command execution, but the underlying scripts still map to the weekly build-up. This makes it possible to run the whole system or study one layer at a time.

9 Repository Map

The repository is organized so that each conceptual layer has a small code area. The storage code lives under `src/mlstore_lite/storage/`. This is where the WAL, memtable, SSTable-like files, compaction, and recovery logic are implemented. The replication code

lives under `src/mlstore_lite/replication/`, where nodes are grouped into leader- follower clusters. The sharding code lives under `src/mlstore_lite/sharding/`, where the consistent hash ring and sharded cluster wrapper route keys to the correct replica group.

The data processing layers are split into batch and stream folders. The batch engine under `src/mlstore_lite/batch/` computes features from finite event lists. The stream code under `src/mlstore_lite/stream/` stores append-only events, tracks consumer offsets, and computes windowed feature updates. The integration layer connects these parts so that batch and stream outputs go into the same sharded replicated store.

The AI and inspection layers are also separated. The feature-serving and model logic live under `src/mlstore_lite/ai/`. Training helpers for the RetailRocket-style event data live under `src/mlstore_lite/training/`. The metadata, lineage, and quality checks live under `src/mlstore_lite/catalog/` and related modules. Finally, `src/mlstore_lite/experiments/` contains the runnable demos for evaluation, inference, recommendation, and the final end-to-end run.

The tests under `tests/` are important because they keep the project from being only a narrative report. They check storage behavior, replication, sharding, batch processing, stream processing, observability, AI inference, and the later inspection layer. The documentation under `docs/weekly-notes/` acts as a learning diary, while this final report gives the compact submission version of the same story.

10 Limitations and Future Work

The most important limitation is that the project is local. Nodes are Python objects and directories, not independent networked processes. The project does not include automatic consensus, real leader election, background repair, transaction isolation, a model registry, HTTP serving, or production monitoring. The AI layer is intentionally small, and the recommender is not trained as a large neural network.

Future work could revisit the repository using the newer DDIA edition, add networked nodes, implement a small consensus experiment, or build mini-projects inspired by *System Design Interview*. On the AI side, the tiny attention model could be replaced by a PyTorch Transformer, a two-tower retrieval model, or a Mamba-style state-space model for longer histories.

11 Conclusion

MLStore-Lite implements a compact data infrastructure prototype for ML-style features. It starts from a single-node storage engine and gradually adds replication, sharding, batch processing, stream processing, observability, feature serving, sequence-based recommendation, and inspection tools.

The final architecture can be summarized as:

```
events -> quality checks -> features -> sharded store
       -> inference/recommender -> lineage -> prediction logs
```

The value of the project is not raw performance. The value is that the internals are small enough to inspect, while still connecting to the architecture of real data-intensive and ML infrastructure systems.

The main learning outcome is that abstract systems ideas become much clearer when implemented end to end. Storage, replication, sharding, streams, and model serving are often discussed as separate topics, but in an ML system they depend on each other. The project helped me see that ML infrastructure is not only about choosing a model. It is also about how data is written, copied, routed, computed, served, checked, and explained after a prediction is made.

Key Terms

DDIA *Designing Data-Intensive Applications*, the main book used as architectural inspiration.

Storage engine
The layer that writes and reads key-value data on one local node.

Key-value store
A storage model where every record is addressed by a key and returns a value.

WAL Write-ahead log: durable append-only record written before in-memory state is updated.

Memtable In-memory table for recent writes before they are flushed to disk.

SSTable-like file
Immutable sorted file inspired by Sorted String Tables in log-structured storage.

Compaction
Background merge process that combines files and removes overwritten values.

Node One local storage instance backed by its own directory.

Replica A copy of the same shard data stored on another node.

RF Replication factor. In this project RF=3 means one leader and two followers per shard.

Leader Replica that receives writes first in the leader-follower setup.

Follower Replica that copies writes from the leader.

Failover Switching to another replica when the current leader is unavailable.

Shard Logical partition of the key space. A shard owns some keys, not one specific file.

Consistent hashing
Routing method that maps keys and shards onto a hash ring.

Virtual node
Extra hash-ring position used to spread keys more evenly across shards.

Feature Derived value used by an ML model, such as click count or purchase count.

Feature key
Human-readable key naming a feature, such as a user's click count.

Feature store
Shared place where computed features are stored and served to models.

Batch processing
Processing a finite dataset, usually historical data already available.

Map-shuffle-reduce
Pattern where records are mapped, grouped by key, and reduced into final values.

Stream processing
Processing events as they arrive over time.

Event log Append-only sequence of events read by consumers.

Offset Consumer position inside an event log.

Tumbling window
Fixed-size time window used to group recent stream events.

Observability
Logging, timing, and recording enough information to inspect system behavior.

JSON-lines File format where each line is a separate JSON object.

Lineage Record of which inputs were used to create a feature or prediction.

Quality check
Validation step that accepts or rejects input events before processing.

Inference Running a model on served features or event histories to produce a prediction.

Sequential recommender
Model that uses the order of user events, not only aggregate counts.

Token Symbol used by the recommender, such as `event:view` or `item:laptop`.

Attention Mechanism that assigns different weights to parts of a user history.

RetailRocket dataset
Kaggle ecommerce event dataset supported by the recommender scripts.

References

[1] RetailRocket, "E-commerce dataset." Kaggle dataset, 2017. Available at https://www.kaggle.com/datasets/retailrocket/ecommerce-dataset.

[2] M. Kleppmann, *Designing Data-Intensive Applications*. O'Reilly Media, 2017.

[3] W.-C. Kang and J. McAuley, "Self-attentive sequential recommendation," *arXiv preprint arXiv:1808.09781*, 2018.

[4] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," *arXiv preprint arXiv:2312.00752*, 2023.